

Lecture 14: Advanced Binary Search (Rotated Arrays & Mathematical Search Spaces)

1. Core Paradigm: Non-Standard Monotonic Spaces

Standard binary search requires a perfectly sorted, single continuous monotonic line. However, binary search can be applied to more complex structures if we can identify a strict mathematical condition that reliably divides the search space into two definitive halves (True/False regions).

Problem-Solving Deconstruction

Problem 1: Find Pivot in a Rotated Sorted Array

1. Problem Statement & Constraints: Given a strictly sorted array that has been cyclically rotated (e.g., `[1, 2, 3, 7, 9]` becomes `[7, 9, 1, 2, 3]`), find the "pivot" index. In this context, the pivot is defined as the absolute minimum element in the array (the starting point of the original unrotated array).

2. Core Intuition: If you graph the values of a rotated sorted array, they form two distinct, disconnected ascending lines.

- **Line 1 (Left side):** All elements in this line are strictly greater than or equal to `arr[0]`.
- **Line 2 (Right side):** All elements in this line are strictly less than `arr[0]`. By evaluating `arr[mid] >= arr[0]`, we instantly know which line we are standing on. If True, we are on Line 1, and the pivot (the dip) must lie to our right. If False, we are on Line 2 (or exactly at the pivot), so the pivot must be at `mid` or to our left.

3. Algorithmic Steps:

1. Initialize `s = 0` and `e = n - 1`.
2. Open a `while` loop bound strictly by `s < e`. (Do not use `<=` to prevent infinite loops when manipulating `e = mid`).
3. Calculate `mid = s + (e - s) / 2`.
4. Evaluate line position: `if (arr[mid] >= arr[0])`.
5. If True, we are on the first ascending line. Move right: `s = mid + 1`.
6. If False, we are on the second ascending line. Move left: `e = mid`. (Notice `mid` is included because it might be the exact pivot).
7. When the loop breaks, `s` and `e` converge at the pivot. Return `s`.

4. Dry Run:

Input: `arr = [7, 9, 12, 3, 5]`

1. `s = 0, e = 4. mid = 2 (val 12).`
2. Evaluate: `12 >= 7 (True)`. We are on Line 1. Pivot is to the right. Update `s = mid + 1 = 3.`
3. `s = 3, e = 4. mid = 3 (val 3).`
4. Evaluate: `3 >= 7 (False)`. We are on Line 2. Update `e = mid = 3.`
5. `s = 3, e = 3`. Loop `s < e` terminates. Return `3`. (Index 3 is the pivot value `3`).

5. Complexity Analysis Table:

Metric	Complexity	Justification
Time Complexity	$O(\log N)$	The search space is strictly halved on every iteration.
Space Complexity	$O(1)$	No extra memory allocation required.

C++

```
#include<iostream>
using namespace std;

int getPivot(int arr[], int n) {
    int s = 0;
    int e = n - 1;
    int mid = s + (e - s) / 2;

    while (s < e) {
        // Condition defining the first monotonic line
        if (arr[mid] >= arr[0]) {
            s = mid + 1;
        }
        // Condition defining the second monotonic line
        else {
            e = mid;
        }
        mid = s + (e - s) / 2;
    }
    return s;
}
```

Problem 2: Search in a Rotated Sorted Array

1. **Problem Statement & Constraints:** Given a rotated sorted array and a target integer `k`, return the index of `k`. If not found, return `-1`.
2. **Core Intuition:** A rotated sorted array is simply two independent, perfectly sorted arrays stitched together. If we locate the pivot (using the logic from Problem 1), we perfectly identify the boundaries of these

two sorted halves. We can then evaluate which half K belongs to and execute a standard Binary Search on that specific half.

3. Algorithmic Steps:

1. Call `getPivot(arr, n)` to find the minimum element's index.
2. **Boundary Check:** Check if K falls within the range of the second sorted line. If $K \geq \text{arr}[\text{pivot}]$ AND $K \leq \text{arr}[n-1]$, target the second half.
3. Execute standard `binarySearch(arr, pivot, n-1, K)`.
4. If False, the element must logically reside on the first sorted line. Target the first half.
5. Execute standard `binarySearch(arr, 0, pivot-1, K)`.

4. Complexity Analysis Table:

Metric	Complexity	Justification
Time Complexity	$O(\log N)$	Finding the pivot takes $O(\log N)$. Searching the isolated half takes $O(\log N)$. Total: $O(2 \log N) \rightarrow O(\log N)$.
Space Complexity	$O(1)$	Strictly constant memory.

Problem 3: Square Root using Binary Search (Floor Value)

1. Problem Statement & Constraints: Given a non-negative integer N , mathematically compute and return its square root. If the square root is a decimal, return only the integer (floor) part. Do not use built-in exponent functions.

2. Core Intuition: The square root of any number N must fall within the range of 0 to N . The sequence $0, 1, 2, \dots, N$ is strictly monotonic. Therefore, we can apply Binary Search on the *answer space* itself.

- **The Overflow Trap:** Squaring the `mid` pointer (`mid * mid`) for large inputs will easily exceed the 32-bit `INT_MAX` limit. We must enforce a 64-bit space by declaring variables as `long long int`.

3. Algorithmic Steps:

1. Initialize search space: `s = 0` and `e = n`. Declare `ans = 0`.
2. Open `while(s <= e)`. Compute `long long int mid = s + (e - s) / 2`.
3. Compute the square: `long long int square = mid * mid`.
4. **Perfect Match:** If `square == n`, return `mid` immediately.
5. **Under-shoot (Potential Answer):** If `square < n`, `mid` is a valid floor candidate. Record `ans = mid` and search the right half for a larger valid integer: `s = mid + 1`.

6. **Over-shoot:** If `square > n`, `mid` is invalid. Search the left half: `e = mid - 1`.
7. Return `ans` when the loop halts.

4. Complexity Analysis Table:

Metric	Complexity	Justification
Time Complexity	$O(\log N)$	Binary search applied to a continuous range from 0 to N .
Space Complexity	$O(1)$	Computations operate using scalar trackers.

C++

```
class Solution {
public:
    long long int binsqrt(int n) {
        int s = 0;
        int e = n;
        long long int mid = s + (e - s) / 2;
        long long int ans = 0;

        while (s <= e) {
            // Guarding against integer overflow during multiplication
            long long int square = mid * mid;

            if (square == n) {
                return mid;
            }
            if (square < n) {
                ans = mid;           // Store potential floor answer
                s = mid + 1;         // Seek larger candidate
            }
            else {
                e = mid - 1;         // Too large, seek smaller
            }
            mid = s + (e - s) / 2;
        }
        return ans;
    }

    int mySqrt(int x) {
        return binsqrt(x);
    }
};
```

Problem 4: Expanding Square Root to Decimal Precision

1. Problem Statement & Constraints: Extend the functionality of the previous algorithm to calculate the square root up to a specific number of decimal places (precision).

2. Core Intuition: Once the integer floor is isolated (e.g., $\sqrt{37} \approx 6$), we can systematically add precision using nested additive testing. We iterate and test adding 0.1 until the square overshoots, then switch to adding 0.01, then 0.001, etc.

3. Algorithmic Steps:

1. Obtain the integer floor using the Binary Search function (`tempSol = sqrtInteger(n)`).
2. Define a function `morePrecision(n, precision, tempSol)` .
3. Initialize `double factor = 1` and `double ans = tempSol` .
4. Run an outer `for` loop matching the requested precision .
5. Mathematically shift the factor constraint: `factor = factor / 10` .
6. Run an inner `for` loop testing incremental additive steps: `for (double j = ans; j * j < n; j = j + factor)` .
7. Continually update `ans = j` inside the inner loop.
8. Return the highly precise `ans` .